SOLUTION BRIEF / NOVEMBER 2026

A cited review triage assistant

Problem 3 prototype | Evidence-first RAG over public Kuku FM reviews

Input	Scope
185 English Google Play reviews, India storefront; 200 requested, empty/duplicate text removed.	One app listing, not every show. Reviews are self-selected and do not verify reported events.

Problem and hypothesis

Product and content teams may need to find recurring complaints and requests across listener feedback. The hypothesis is that a teammate can ask a question in plain language and quickly inspect relevant reviews with citations. This is a testable workflow hypothesis; it has not been validated with Kuku FM employees.

RAG pipeline implemented

1. Plan the question. LLM summary mode asks Groq for a few short search rewrites in JSON. Evidence-only mode uses transparent rule-based expansions. Malformed output or a planning failure falls back to those rules.

2. Keep parent reviews; index child passages. Each full review is the parent evidence record. Reviews longer than 48 words become overlapping 48-word passages with 12-word overlap; shorter reviews remain one passage. The dataset creates 208 children.

3. Load stored bi-encoder vectors. The checked-in 208 x 384 float32 matrix was created with sentence-transformers/all-MiniLM-L6-v2 and keyed by a digest of model plus child text. scripts/build_review_embeddings.py regenerates it. This skips re-embedding the corpus, but the encoder still loads to vectorize each new question.

4. Retrieve by cosine similarity. The bi-encoder embeds the question and any rewrites. Scores are max-pooled across query variants; the best child per parent is kept. Up to 30 distinct parent reviews continue to ranking.

5. Rerank with a cross-encoder. cross-encoder/ms-marco-MiniLM-L6-v2 scores each child against the original question. The system drops candidates more than 5 score points below the best and keeps up to 8. Scores are relative logits, not probabilities.

6. Return evidence or synthesize. Evidence-only mode returns full parent reviews, IDs, dates, ratings, and scores without LLM calls. LLM summary mode sends those full reviews to Groq for a cited answer, then asks Groq to audit support and citations. Planning, synthesis, and audit can mean three sequential API calls. The audit is not a formal validator.

Two ways to use it

Evidence only	LLM summary
No API calls; rule-based query expansion; inspect retrieved reviews directly.	Groq query planning, cited synthesis, and citation-support audit; uses the configured Groq key and quota.

What the prototype demonstrates

Evidence for the workflow, known failures, and a responsible path to broader coverage

What we checked

The repository has 20 automated unit tests covering query rewrites and fallback, provider configuration, retrieval ordering and stable citations, cross-encoder filtering, offline mode, and pipeline failure paths. Model and API calls are mocked where appropriate, so these tests do not measure live retrieval quality. raw_outputs/ preserves four offline runs and one captured Groq payment/refund run. The Groq example cites individual reports and avoids presenting them as prevalence. The offline playback and reliability examples contain weak or generic matches.

Known limitations

- The 185 reviews are English-only, self-selected, and limited to one app listing; results are not a representative listener survey.

- There is no labeled relevance benchmark or measured Recall@k, nDCG, citation accuracy, latency, or usefulness score. A plausible answer can still cite weak evidence; inspect source reviews and keep a human decision-maker.

- Render Free reported an instance failure after exceeding 512 MB RAM. The stored corpus vectors skip document re-embedding but do not remove runtime loading of the query encoder and cross-encoder.

- LLM summary mode sends question and retrieved review text to Groq. The public app uses a server-side GROQ_API_KEY when configured, so visitors share the owner's quota. Keep the key out of GitHub and monitor provider usage.

How this could scale to every show

The current data is app-store feedback, not show-level feedback. First connect a permitted source that includes show/episode identifiers. Store each item with show_id, source, locale, timestamp, and a stable ID; ingest incrementally and deduplicate. Preserve parent-child links and metadata. As volume grows, move the in-memory vectors to a vector index and filter by show, language, and time before reranking. Evaluate a labeled query/review set per show using Recall@k, nDCG@k, citation support, abstention quality, latency, and cost. Add access controls, retention rules, and human review. These scale-up steps are proposed, not implemented.

Prototype and reproduction

Project: GitHub repository (parent-child-retrieval branch) | Raw runs: raw_outputs folder. The README has terminal steps. Evidence-only CLI: **python agentic_rag.py "What do reviewers say about payments?" --offline --debug**. For summaries, copy .env.example to .env, add your own GROQ_API_KEY, and omit --offline. Real keys are never committed.